12.788

RAVULA SHASHANK REDDY - EE25BTECH11047

October 12, 2025

Question

$$\mathbf{L} = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & 0 \\ -1 & 0 & -1 & 1 \end{pmatrix}$$

Which of the following are TRUE?

- The matrix $\mathbf{L}$ is row equivalent to $\begin{pmatrix} 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & 0 \\ -1 & 0 & -1 & 1 \end{pmatrix}$
- The linear system $\mathbf{Lx} = \mathbf{b}$ has a solution for all $\mathbf{b}$
- For $\mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$, the system $\mathbf{Lx} = \mathbf{b}$ has a solution
- Rank of the matrix $\mathbf{L}$ is 3

Solution

Given:

$$\mathbf{L} = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & 0 \\ -1 & 0 & -1 & 1 \end{pmatrix} \tag{1}$$

Row reduction (Option a):

$$R_1 \to R_1 + R_2 + R_3 + R_4 \implies \mathbf{L} \sim \begin{pmatrix} 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & 0 \\ -1 & 0 & -1 & 1 \end{pmatrix} \tag{2}$$

Rank of $\mathbf{L}$ (Option d):

$$\text{One row is dependent, remaining 3 independent} \implies \text{rank}(\mathbf{L}) = 3 \tag{3}$$

System $\mathbf{Lx} = \mathbf{b}$ for all $\mathbf{b}$ (Option b):

$$\text{Rank}(\mathbf{L}) = 3 < 4 \implies \text{Solution will not be there for all b} \tag{4}$$

Solution

Dependence property(Option c):

$$R_1 + R_2 + R_3 + R_4 = \mathbf{0} \tag{5}$$

$$\implies (R_1 + R_2 + R_3 + R_4)\mathbf{x} = 0 \tag{6}$$

$$\implies R_1\mathbf{x} + R_2\mathbf{x} + R_3\mathbf{x} + R_4\mathbf{x} = b_1 + b_2 + b_3 + b_4 = 0 \tag{7}$$

$$\text{For } \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \quad b_1 + b_2 + b_3 + b_4 = 4 \neq 0 \implies \text{no solution} \tag{8}$$

C Code

```c
    #include <stdio.h>
#include <math.h>

#define N 4
#define EPS 1e-9 // Tolerance for floating-point comparison

int main() {
    double L[N][N] = {
        {3, -1, -1, -1},
        {-1, 2, -1, 0},
        {-1, -1, 3, 0},
        {-1, 0, -1, 1}
    };
    double b[N] = {1, 1, 1, 1};
    double A[N][N];
    int i, j, k;

    // Copy L to A (to preserve original)
```

C Code

```
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            A[i][j] = L[i][j];

    // Step 1: Row reduction to find rank
    int rank = N;
    for (i = 0; i < rank; i++) {
        // If diagonal element is zero, try to swap with a lower
            row
        if (fabs(A[i][i]) < EPS) {
            int swap_row = i + 1;
            while (swap_row < rank && fabs(A[swap_row][i]) < EPS)
                swap_row++;
            if (swap_row == rank) {
                rank--;
                for (k = 0; k < N; k++)
                    A[k][i] = A[k][rank];
                i--;
```

C Code

```
            continue;
        }
        // Swap rows
        for (j = 0; j < N; j++) {
            double temp = A[i][j];
            A[i][j] = A[swap_row][j];
            A[swap_row][j] = temp;
        }
    }

    // Eliminate below
    for (j = i + 1; j < N; j++) {
        double factor = A[j][i] / A[i][i];
        for (k = 0; k < N; k++)
            A[j][k] -= factor * A[i][k];
    }
}
```

C Code

```
    // Count non-zero rows
    int non_zero_rows = 0;
    for (i = 0; i < N; i++) {
        int non_zero = 0;
        for (j = 0; j < N; j++) {
            if (fabs(A[i][j]) > EPS) {
                non_zero = 1;
                break;
            }
        }
        if (non_zero) non_zero_rows++;
    }
    rank = non_zero_rows;

    // Step 2: Check for zero row after adding all rows
    double R1[4];
    for (j = 0; j < 4; j++)
        R1[j] = L[0][j] + L[1][j] + L[2][j] + L[3][j];
```

C Code

```
    int zero_row = 1;
    for (j = 0; j < 4; j++)
        if (fabs(R1[j]) > EPS)
            zero_row = 0;

    // Step 3: Compatibility condition
    double sum_b = 0;
    for (i = 0; i < N; i++)
        sum_b += b[i];

    // Step 4: Print results
    if (zero_row)
        printf(Option a) TRUE\n);
    else
        printf(Option a) FALSE\n);
    if (rank == 4)
        printf(Option b) TRUE\n);
```

C Code

```
    else
        printf(Option b) FALSE\n);

    if (fabs(sum_b) < EPS)
        printf(Option c) TRUE\n);
    else
        printf(Option c) FALSE\n);

    if (rank == 3)
        printf(Option d) TRUE\n);
    else
        printf(Option d) FALSE\n);

    // Optional: Print computed rank
    printf(\nComputed rank of L = %d\n, rank);

    return 0;
}
```

Python Direct

```
    import numpy as np

# Define matrix L and vector b
L = np.array([
    [3, -1, -1, -1],
    [-1, 2, -1, 0],
    [-1, -1, 3, 0],
    [-1, 0, -1, 1]
], dtype=float)

b = np.array([1, 1, 1, 1], dtype=float)

# Step 1: Make R1 = R1 + R2 + R3 + R4
R1 = L[0] + L[1] + L[2] + L[3]

# Check if the new R1 is a zero row
zero_row = np.allclose(R1, np.zeros(4))

# Step 2: Rank of L
```

Python Direct

```
rank = np.linalg.matrix_rank(L)

# Step 3: Compatibility condition
sum_b = np.sum(b)

# Step 4: Print results
if zero_row:
    print(Option a) TRUE)
else:
    print(Option a) FALSE)

if rank == 4:
    print(Option b) TRUE)
else:
    print(Option b) FALSE)
```

Python Direct

```
if np.isclose(sum_b, 0):
    print(Option c) TRUE)
else:
    print(Option c) FALSE)

if rank == 3:
    print(Option d) TRUE)
else:
    print(Option d) FALSE)
```